

CAMPUSOS • 5-PERSON ROLE MAP

React + Django + PostgreSQL + 2 AI Tracks

The BPUT brief asks for end-to-end workflows, admin visibility, recurring-issue insight and accessibility; optional intelligence includes complaint routing, recurring-issue detection, facility prediction and a chatbot.

1. Role map at a glance

2. Backend + Authentication Developer

Mission: secure API boundary between React, database and AI services.

Tech / tools

- Django + Django REST Framework
- JWT
- Permissions/RBAC
- CORS
- Environment variables

Exact responsibilities

- Register/login/refresh/logout.
- Student vs Admin permissions.
- Complaint, leave, gate-pass, certificate, notice and related APIs.
- Input validation, error handling and status transitions.
- AI integration endpoints.

Flow

- React → JWT → protected endpoint → permission check → business logic → DB/AI → JSON.

Deliverables

- Auth APIs
- Core workflow APIs
- RBAC
- Validation
- AI gateway endpoints
- Audit/status transitions

Learn

- Django/DRF, serializers, views, URLs, JWT, permissions, HTTP status codes, API testing.

Integration

- Person 1 consumes APIs. Person 3 owns models/migrations. Person 4/5 provide AI functions.

Test / demo

- Test unauthorized and forbidden requests, bad input, token expiry and every end-to-end workflow.

3. Database + Backend Developer

Mission: create the source of truth and reliable data for workflows and AI.

Tech / tools

- PostgreSQL
- Django ORM
- Migrations
- Indexes/constraints
- Seed/demo data

Exact responsibilities

- Design Student/Admin/User relationships.
- Complaint, category, status, timestamps, leave/gate-pass, certificates, notices, mess/hostel data.
- Store AI results and risk/cluster outputs.
- Prepare realistic seed data.
- Create efficient dashboard queries.

Flow

- API → Django ORM → PostgreSQL. AI reads clean data → result stored → dashboard reads result.

Deliverables

- ERD/schema
- Migrations
- Seed dataset
- Dashboard queries
- AI-ready datasets

Learn

- Relational modeling, keys/relationships, Django ORM, migrations, basic SQL and indexes.

Integration

- Person 2 consumes models. Person 4 uses complaint/doc data. Person 5 uses historical structured data. Person 1 only sees API JSON.

Test / demo

- Validate foreign keys/duplicates. Verify dashboard counts. Keep DB credentials server-side.

4. AI Model 1 — NLP + RAG

Mission: own unstructured language, meaning-based retrieval and the campus assistant.

Tech / tools

- LLM API
- Sentence embeddings
- Cosine similarity
- FAISS or ChromaDB — choose one
- Prompt templates
- Structured JSON

Exact responsibilities

- Complaint summarization/extraction.

- Embeddings for complaints and documents.
- Semantic search.
- RAG over policies/notices/FAQs.
- Chatbot with evidence/source.
- Optional few-shot prompting.

Flow

- Complaint text → LLM → structured fields. Documents → chunks → embeddings → vector DB. Question → Top-K context → LLM → answer + evidence.

Deliverables

- Complaint analyzer
- Embedding service
- Semantic search
- RAG ingestion/retrieval
- Chatbot

Learn

- LLM basics, prompts, JSON output, hallucination, embeddings, similarity, chunking, vector DB, RAG.

Integration

- Person 2 secures AI endpoints. Person 3 supplies data. Person 1 renders results. Person 5 may reuse embeddings for clustering.

Test / demo

- Test known questions. Check retrieved evidence. Handle no-result cases. Use fixed JSON schemas.

5. AI Model 2 — ML Analytics + Prediction

Mission: own structured-data ML and admin intelligence.

Tech / tools

- Python
- Pandas/NumPy
- scikit-learn
- Embeddings from Person 4 for complaint clustering

Exact responsibilities

- Classification: complaint → department/category.
- Clustering: similar complaints → recurring incident.
- Anomaly: unusual complaint/facility patterns.
- Risk prediction: structured features → probability.
- What-if: modify selected features → rerun model → compare.

Flow

- Complaint → classifier. Embeddings → K-Means/DBSCAN. Historical values → Z-score/Isolation Forest. Features → Logistic Regression/Random Forest → risk.

Deliverables

- Classification endpoint

- Recurring-issue pipeline
- Anomaly endpoint
- Risk endpoint
- What-if endpoint

Learn

- Classification, train/test split, precision/recall/F1, K-Means, DBSCAN basics, anomaly detection, Logistic Regression, Random Forest, probability vs prediction.

Integration

- Person 3 provides data. Person 4 provides embeddings. Person 2 exposes protected endpoints. Person 1 visualizes results.

Test / demo

- Use seeded demo data. Keep train/test separation. Explain false positives/negatives. Label risk as a signal, not certainty.

6. Shared architecture

Suggested API contract

POST /api/auth/login

POST /api/complaints

GET /api/complaints

POST /api/ai/complaint-analyze

POST /api/ai/search

POST /api/ai/chat

GET /api/admin/insights

POST /api/ai/classify

POST /api/ai/cluster

POST /api/ai/anomaly

POST /api/ai/risk

POST /api/ai/what-if

7. One complaint — complete team workflow

- Student enters: “Water leaking in Block A for 9 days.”
- Person 1: React sends the complaint.
- Person 2: Django authenticates, validates and creates the complaint.
- Person 3: PostgreSQL stores complaint + timestamps.
- Person 4: LLM extracts summary/urgency; embedding represents meaning.
- Person 5: classifier routes it; clustering checks recurring complaints.
- Backend stores AI results.
- Person 1 shows category, recurring incident, ageing and evidence on the admin dashboard.
- Admin makes the final operational decision.

8. DEMO-ONLY EXECUTION PLAN

Scope: Build only enough to demonstrate the system working in 2–3 minutes. The remaining presentation time is used to explain architecture, AI roles and impact.

What the prototype MUST demonstrate

- Student login → student dashboard.
- Student submits one realistic complaint: “Water leaking in Block A for 9 days.”
- Complaint is saved and visible in request history.
- AI 1 analyzes it: category + summary + urgency + confidence.
- AI 2 groups similar complaints into a recurring incident such as “Water Failure”.
- Admin dashboard shows the complaint, recurring incident, affected block and status.
- RAG/chatbot answers one campus-policy or notice question with visible source/evidence.
- Admin sees an AI risk/anomaly signal and makes the final decision manually.

Do NOT build these for the demo

- Complete every student feature in the problem statement.
- A production-grade ML training pipeline.
- Your own LLM or embedding model.
- Complex microservices or DevOps infrastructure.
- Full notification infrastructure.
- Advanced animations or unnecessary UI polish.
- Separate demo screens that are not connected to the main workflow.

2–3 minute live-demo path

1. Login as Student.
2. Open Complaints → Submit Complaint.
3. Enter the prepared water-leak complaint.
4. Click Analyze → show category, summary, urgency and confidence.
5. Switch to Admin → show the recurring “Water Failure” incident.
6. Open AI Assistant → ask one campus-policy/notice question → show source evidence.
7. Show the AI risk/anomaly card.
8. Finish with the admin manually deciding the action.

5-minute presentation structure

The single story to rehearse

- Student reports a water failure.
- Backend stores it securely.
- AI 1 understands the text.
- AI 2 finds the recurring pattern.
- Admin sees the operational impact.
- RAG answers the relevant policy question.

→ Admin decides what action to take.

CORE IDEA Person 4 owns language + retrieval. Person 5 owns structured-data ML + analytics. This prevents AI overlap.

PERSON	ROLE	OWNS	NOT OWNED
1	Frontend	React, dashboards, API integration	AI logic, DB schema
2	Backend + Auth	Django/DRF, JWT, RBAC, security	React UI, ML training
3	DB + Backend	PostgreSQL, models, data, queries	Auth policy, ML training
4	AI 1 — NLP/RAG	LLM, embeddings, search, RAG, chatbot	CRUD/API ownership
5	AI 2 — ML	classification, clustering, anomaly, risk, what-if	React/auth ownership

RULE Do not build 11 disconnected AI features. Build one connected complaint workflow and add AI layers around it.

AI CONTRACT Every AI endpoint should return predictable JSON so React never needs to know model internals.

DEMO STORY One complaint should travel through the whole architecture. That proves the five roles are one product.

SCOPE CONTROL If time gets tight, protect complaint routing + recurring incidents + RAG chatbot + admin dashboard. Risk/what-if can remain an extension.

TIME	SECTION	WHAT TO SAY/SHOW
0:00–0:40	Problem	Scattered campus information, manual complaints and slow issue tracking.
0:40–1:20	Solution	CampusOS: Student → Backend → AI → Admin.
1:20–3:50	Live demo	Login → complaint → AI analysis → recurring incident → admin dashboard → RAG answer.
3:50–4:30	Architecture	Explain the 5 roles and AI 1 vs AI 2.
4:30–5:00	Close	AI provides signals/evidence; the admin makes the final operational decision.